



**INTERNATIONAL CAMPUS
OF SCIENCE AND TECHNOLOGY**

International Campus of Science and Technology

Faculty of Computing, Department of IT
HDIT Batch 01

HDIT - DBMS

Project Name: Restaurant Management System - Cafulon

Submitted by:

1. NM. Roshaan Akther
2. A. Washid
3. MAM. Masheef
4. MIF. Fahmidha
5. MNF. Nuha
6. WMCS. Weerakkodi

Submission Date: Jun 9, 2026

OPTIMIZATION REPORT Cafulon

Restaurant Management System

1. Index Analysis

1.1 Existing Indexes

Index Name	Table	Column(s)	Type	Purpose
idx_orders_date	Orders	OrderDate	B-tree	Accelerates date-range queries and daily sales reports
idx_orders_status	Orders	Status	B-tree	Speeds up filtering by order status (Pending/Preparing/Completed)
idx_reservations_date	Reservations	Reservation Date	B-tree	Optimizes daily reservation lookup for hostess
idx_menu_items_category	MenuItems	CategoryID	B-tree	Accelerates JOIN with MenuCategories
idx_payments_order	Payments	OrderID	B-tree	Speeds up payment lookup by order ID

1.2 Justification

- Orders table benefits most from indexing because it is the most queried table — filtered by date (daily reports), status (pending orders count), and joined with multiple tables.
- ReservationDate is queried daily to show "Today's Reservations" — an index eliminates full table scans.
- CategoryID in MenuItems is a foreign key used in every menu listing JOIN — indexing it is standard practice.
- OrderID in Payments is used to look up payments when viewing an order detail page.

2. Generated Columns

The OrderDetails table uses a MySQL generated column for subtotal calculation:

Subtotal DECIMAL(10, 2) GENERATED ALWAYS AS (Quantity * UnitPrice) STORED

Benefits:

- Eliminates application-level calculation logic
- Guarantees consistency between Quantity, UnitPrice, and Subtotal
- The STORED keyword materializes the column, avoiding recalculation on every SELECT
- Reduces storage compared to recomputing on every read (vs. VIRTUAL)

Trade-off: Uses additional disk space (negligible for this dataset size).

3. Query Optimization

3.1 EXPLAIN Analysis

All queries were analyzed using EXPLAIN to verify index usage:

Query	Table Scan Avoided?	Index Used	Comments
Q1: Available Menu Items	Yes	(full scan on 33 rows, acceptable)	Small table; no index needed
Q5: Order Details Join	Yes	idx_orders_date, PKs	All joins use indexed columns
Q6: Customers LEFT JOIN Orders	Yes	PK (CustomerID)	Full table scan on Customers (small)
Q7: Daily Sales Report	Yes	idx_orders_status, idx_orders_date	Both index conditions used
Q8: Top Selling Items	Yes	PKs, FK indexes	GROUP BY and ORDER BY on indexed cols
Q10: Reservation Details	Yes	idx_reservations_date, PKs	All JOIN columns indexed

3.2 JOIN Optimization

- INNER JOIN used where NULL results are never needed (e.g., Order → OrderDetail)
- LEFT JOIN used where NULLs are possible (e.g., Customers having no orders, walk-in orders with no customer)

- Join order follows the schema: smallest result set drives the join

3.3 Aggregate Function Optimization

- COALESCE(SUM(...), 0) prevents NULL results in financial calculations
- ROUND(AVG(...), 2) reduces decimal precision for readability
- COUNT(DISTINCT ...) used where needed to avoid double-counting

4. Normalization Review

The schema was audited for redundancy:

Table	Normal Form	Issues Found
Customer	3NF	None
Employee	3NF	None
MenuCategory	3NF	None
MenuItem	3NF	None
RestaurantTable	3NF	None
Reservation	3NF	None
Order	3NF	None
OrderDetail	3NF	Generated column avoids redundancy
Payment	3NF	None

No partial or transitive dependencies were found in the current schema.

5. Constraint Analysis

5.1 CHECK Constraints Enforced

Table	Constraint	Purpose
Employees	CHECK (Salary > 0)	Prevents invalid salary entries
MenuItems	CHECK (Price > 0)	Menu prices must be positive
RestaurantTables	CHECK (Capacity > 0)	Table capacity must be valid
Reservations	CHECK (PartySize > 0)	Party size must be at least 1
OrderDetails	CHECK (Quantity > 0)	Order quantities must be positive
Payments	CHECK (Amount > 0)	Payment amounts must be positive

5.2 Referential Integrity

Foreign Key	On Delete	On Update	Rationale
MenuItem.CategoryID → MenuCategory	RESTRICT	CASCADE	Prevent deleting a category with items
Reservation.CustomerID → Customer	RESTRICT	CASCADE	Prevent deleting customer with active reservations

Reservation.EmployeeID → Employee	SET NULL	CASCADE	Allow employee deletion without losing reservation
Order.CustomerID → Customer	SET NULL	CASCADE	Walk-in orders have no customer
Order.EmployeeID → Employee	RESTRICT	CASCADE	Every order must have an employee
OrderDetail.OrderID → Order	CASCADE	CASCADE	Deleting an order removes its details
OrderDetail.ItemID → MenuItem	RESTRICT	CASCADE	Prevent deleting a menu item that's in order history
Payment.OrderID → Order	RESTRICT	CASCADE	Payment records are permanent

6. Recommendations

6.1 Additional Indexes

```
-- Composite index for common OrderDetails JOIN pattern
CREATE INDEX idx_order_details_lookup ON OrderDetails(OrderID, ItemID);

-- Index for employee shift queries
CREATE INDEX idx_employees_shift ON Employees(Shift);
```

6.2 Production Optimizations

1. Connection Pooling: Replace per-request connections with a pool (e.g., SQLAlchemy or mysql-connector-pooling)
2. Query Caching: Implement Redis or application-level cache for dashboard statistics
3. Partitioning: Partition Orders table by year for faster historical queries
4. Archive Strategy: Move orders older than 12 months to an archive table to keep the active dataset lean

6.3 Monitoring

- Enable MySQL slow query log (`slow_query_log = 1`) with a threshold of 2 seconds
- Monitor index usage via `SHOW INDEX FROM <table>` and `performance_schema`